Applications of AI Models: Assignment 1

Written Report

Name: Aiyman Akbar
Date: 2026/02/24
Class: Sch-Mgmt 661: Applications of AI Models
Word Count: 1824

1. Introduction

This assignment focuses on building a neural network regression model to predict Airbnb listing prices in Asheville, North Carolina. The dataset is drawn from Inside Airbnb and includes information on listing characteristics, host attributes, and review metrics. The objective is to clean and explore the data, analyze relationships among variables, and develop a predictive model using Keras.

2. Data Exploration and Cleaning

This was the first section of the assignment. Here all the setup was done, data cleaning and preprocessing, so that we can work with the data in the other 2 sections. This assignment in general continued from W3 and a lot of code was reused from there, as was mentioned in the markdown chunks in assignment 1 itself.

First the dataset was loaded up and an initial exploration of its structure and contents, along with the features was done. After getting familiar with that information, only the listings belonging to Asheville, NC were kept from the dataset as those are the only ones needed. Then the features that were needed for the model were kept and the rest were discarded for the purpose of this assignment.

The whole next section was about handling missing values and outliers. First the missing values in each of the columns of the features that are being used, and once found all the rows with the target variable (price) missing were dropped as they cannot be used for prediction. Then imputation was done. Categorical variables like host_is_superhost were imputed using mode as the other two don't work for categoricals, and numerical variables like bedrooms and bathrooms were done using either mean or medium. Then for each of these the corresponding missing values in each of the columns was filed using the imputed values.

Review_scores_rating was a special case as the missing values in that column aren't randomly missing, they mean that this listing hasn't been reviewed yet. Imputing it with the mean would tell the model that unreviewed listings have a high rating which would be false. Instead all the missing values were replaced by 0 and a dummy variable, has_review_score was created. This dummy variable is set to 1 if the listing has a review and 0 if it doesn't.

After that all the data types were checked for all variables and price was a string so it was converted to a float so that the model can use it later for analysis since the model can only use numeric variables. Host_identity_verified and host_is_superhost were also categorical, stored as true/false so they were dummy encoded with 1 to replace yes and 0 to replace no. Room_type was a special case here because it was a multi-category categorical variable, so instead of dummy encoding it had to be one-hot encoded for each of the unique categories in that column.

Now that all the missing values were taken care of, the outliers were dealt with next. The original price distribution was visualized (figure 1 below) and it is extremely skewed and clustered on the left side. After this the top one percent of extreme price values were removed in hopes of getting a better

visualization and removing any values that would impact the model drastically, and then figure 2 was made, which shows a cleaner visualization.

Figure 1:

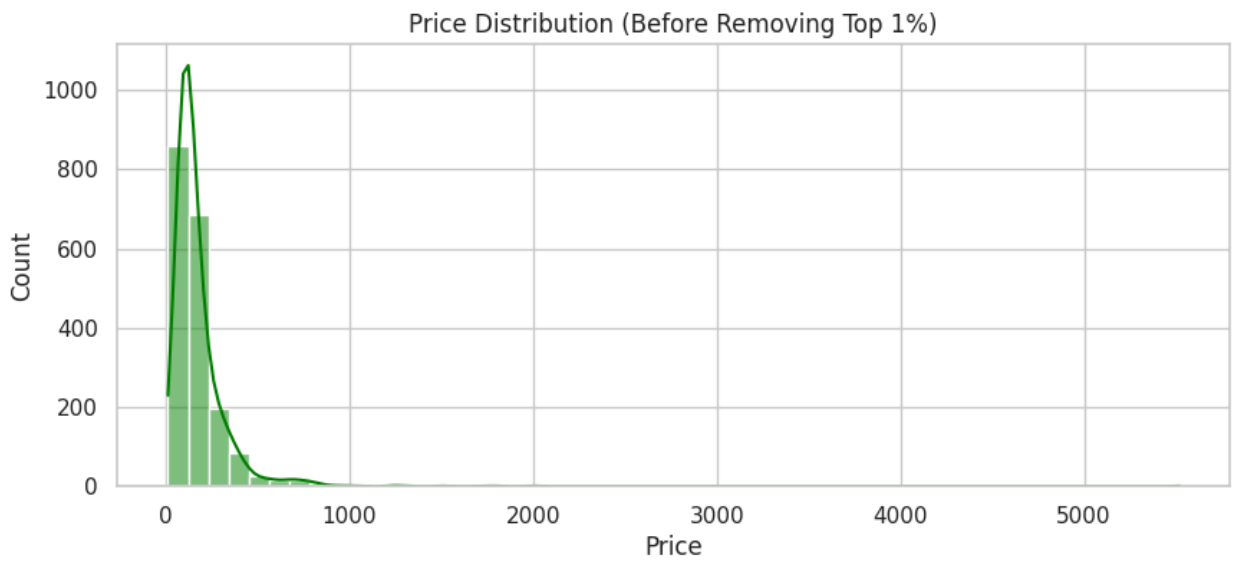
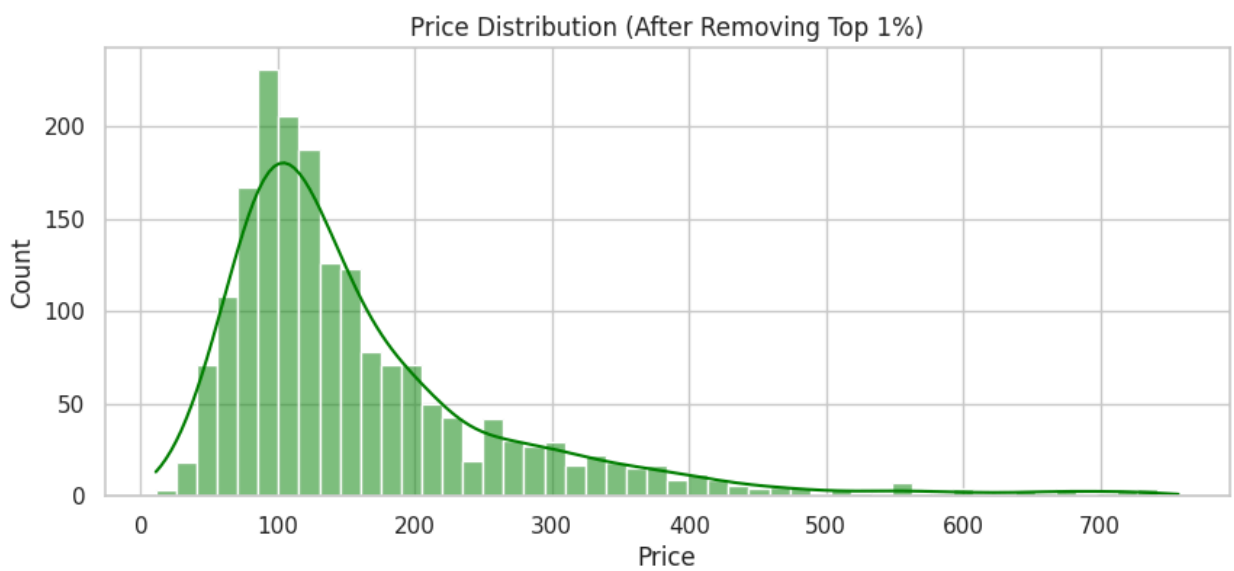


Figure 2:



Once this was done the descriptive statistics were generated based on the cleaned data from the features that were going to be used in the model. Some key takeaways from those statistics include the average listing price is approximately $163, with a relatively large standard deviation of $112, which shows me that the price is still quite variable and can change quite a bit even after dealing with large outliers in section 1. The high mean review score (4.63 out of 5) and the fact that 94% of listings have review scores indicate to me that most properties are well maintained and guests usually have a good stay, thus the high review percentage. However the high review percentage alone doesn't necessarily mean that

they are all positive reviews and that guests usually enjoy their stay, but combined with the mean review score I am led to believe that they did.

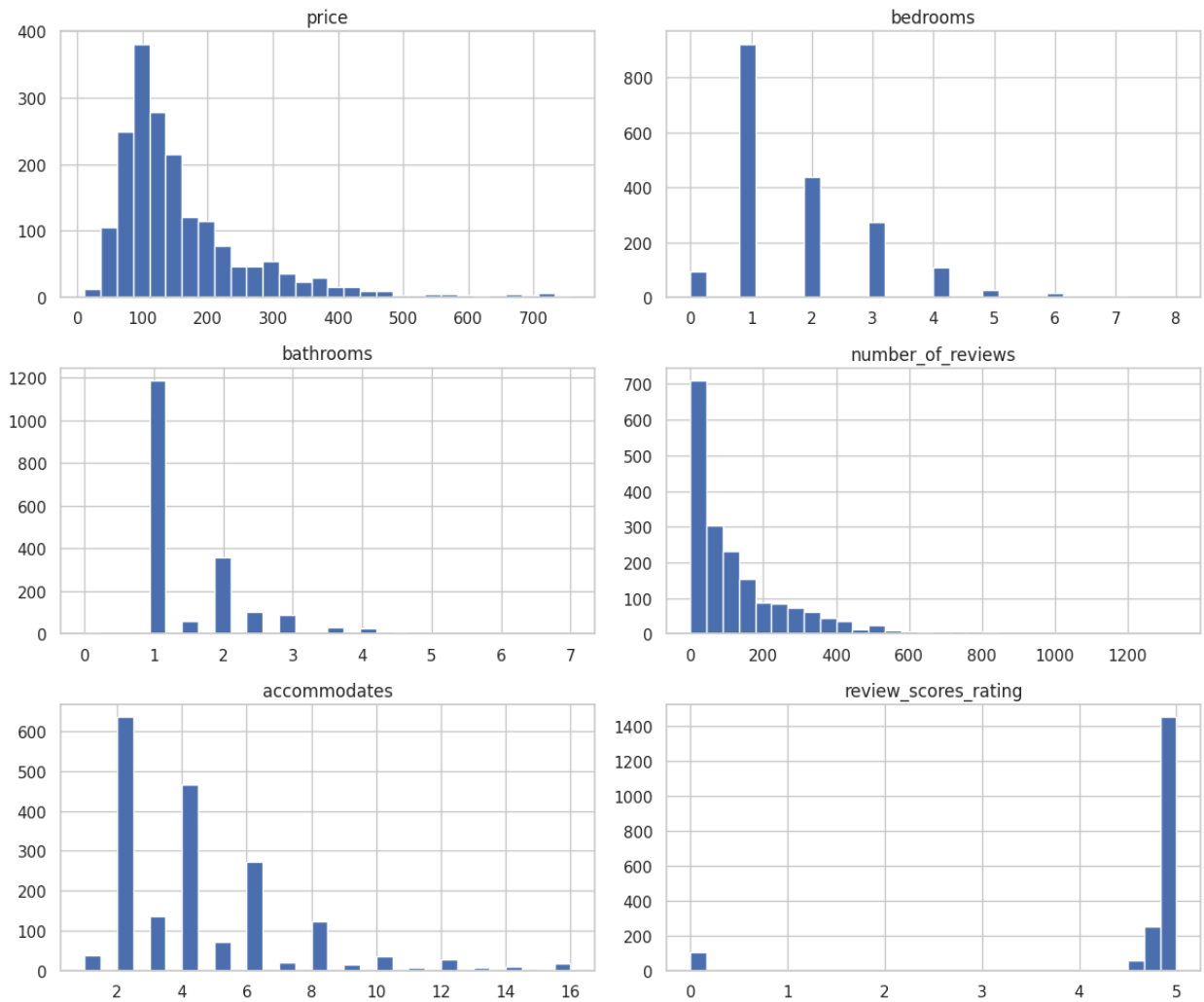
2.1) Suggested Improvements for Data Exploration and Cleaning

There are also some things that could be done in data-cleaning or preprocessing that I think could further improve the models performance. One thing I think could be done is log transform the price. As can be seen in the visualization of the cleaned price distribution the price is skewed very right. Because of this applying a log transformation on the price variable would make the impact of very high priced listing less and later allow the model to learn the patterns more evenly. Something else I think can be done is do the same thing with number_of_reviews. We can see in the table it is very skewed and it would be a good thing to add for the same reason as the log transformation on price. One last thing I want to say is that I think the model would later benefit a lot from combining or removing some of the size related variables. Because variables like bedrooms, bathrooms and accommodates are highly related variables, either combining them or removing a couple of them I think would reduce some redundancy.

3. Exploratory Data Analysis

Once the data was cleaned and prepared to be used, the next step was EDA. There were two components of this section, a collection of exploratory histograms shown in figure 3, and a correlation heatmap which shows the correlation between different features (figure 4).

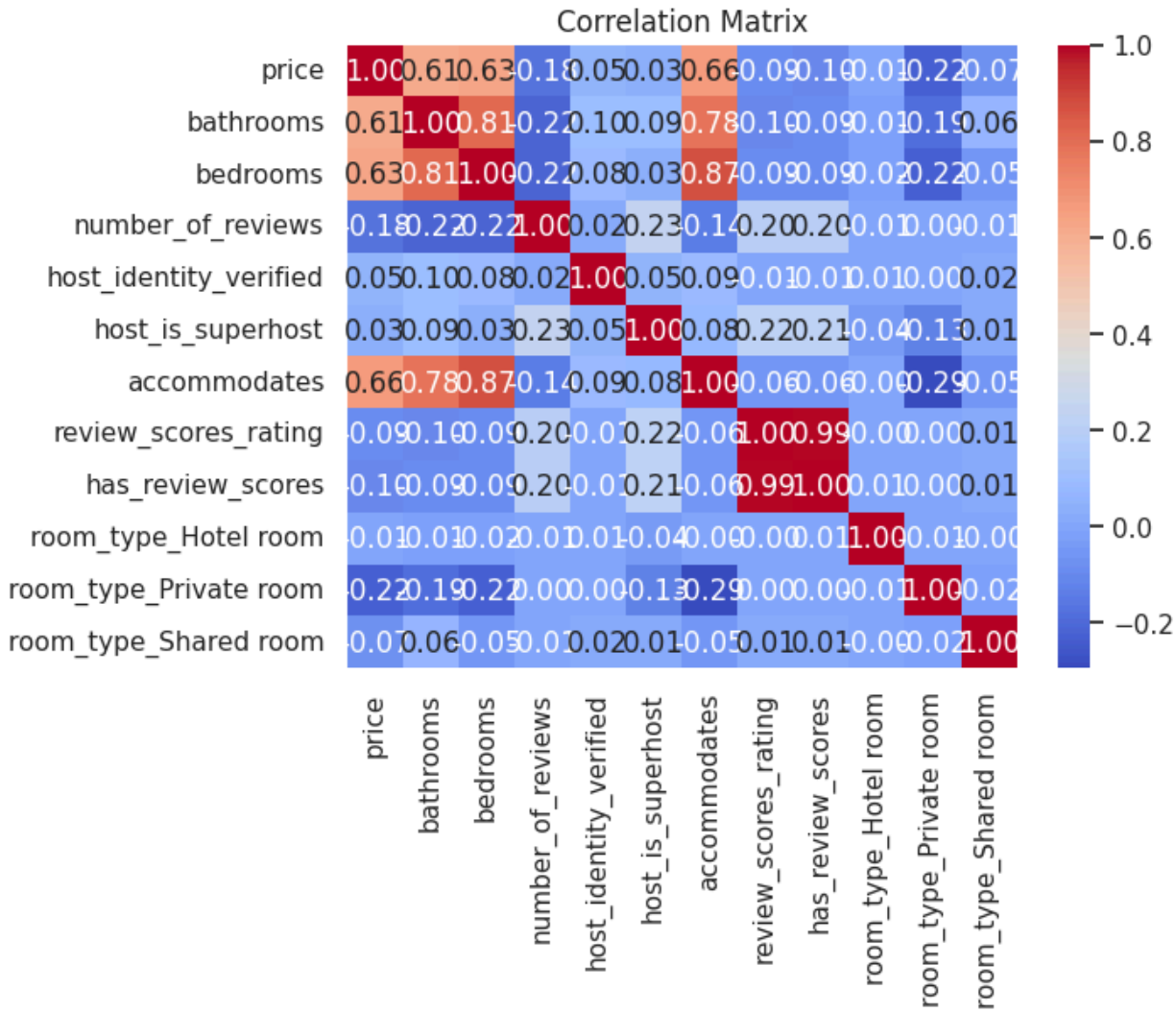
Figure 3:



Like section 1, the histograms show that price and number_of_reviews are extremely right skewed, telling me that most listings are priced somewhere in the middle and have a relatively small number of reviews, while a smaller group has very high values in both. Bathrooms and bedrooms are concentrated around 1–2, suggesting that most listings are smaller properties. The review_scores_rating distribution is heavily concentrated near 5, backing up my observation from section 1 that most places

have high reviews with very few having low ratings. The accommodation variable is centered around 2–6 guests, indicating that there are fewer large capacity properties.

Figure 4:



The correlation matrix (figure 4) reveals that accommodates (0.66), bedrooms (0.63), and bathrooms (0.61) have the strongest positive relationships with price. To me this suggests that property size and capacity are key drivers of listing price. There is a strong correlation between bedrooms and accommodates (0.87) and between bedrooms and bathrooms (0.81), which to me gives some evidence for my idea to remove or combine some of the size related variables due to their strong relationships. Review-related variables show weak correlations with price, meaning ratings and number of reviews don't seem to impact the price that much, which was a little surprising for me.

4. Model Building and Evaluation

This was the last step in assignment 1. After seeing the visualization and descriptive stats from the previous section, the required features were defined and so was the target variable, then divided into training and test sets using train test split. 80% of the data went into the training sets and 20% went into the testing sets. The features were then normalized through a normalization layer to make features that are on vastly different scales like number_of_reviews which is 0-1333 and host_is_superhost which is on a 0-1 scale are not treated disproportionately by the model just because one has bigger raw values. They should both be treated the same, which is why normalization is very important.

Then the last step before compiling the model is building a neural network regression model using Keras Sequential. This was done with one input node per feature, meaning each feature gets one entry point. Next is one hidden layer with 64 nodes ReLu activation. This is the part of the model that takes each input and combines them with the learned weights to look for patterns. The last layer is one output node for the price.

After that the model was compiled with optimizer adam and loss as mean squared error (mse). The loss function of mse measures the average squared difference between predicted and actual prices, penalizing larger errors more heavily. Additionally, metrics like mean absolute error (mae) and mse were tracked to review the model at the end.

After all of this was set up, the model was trained. It trained for 50 epochs (50 times over the entire dataset) with a validation split of 20%. Then based on the result mae over the training was plotted out (figure 5) and so was mse over the training (figure 6). At the end, the final values were test mae at 57.57, test mse at 7129.58, test loss at 7129.58 and R^2 at .47 .

Figure 5:

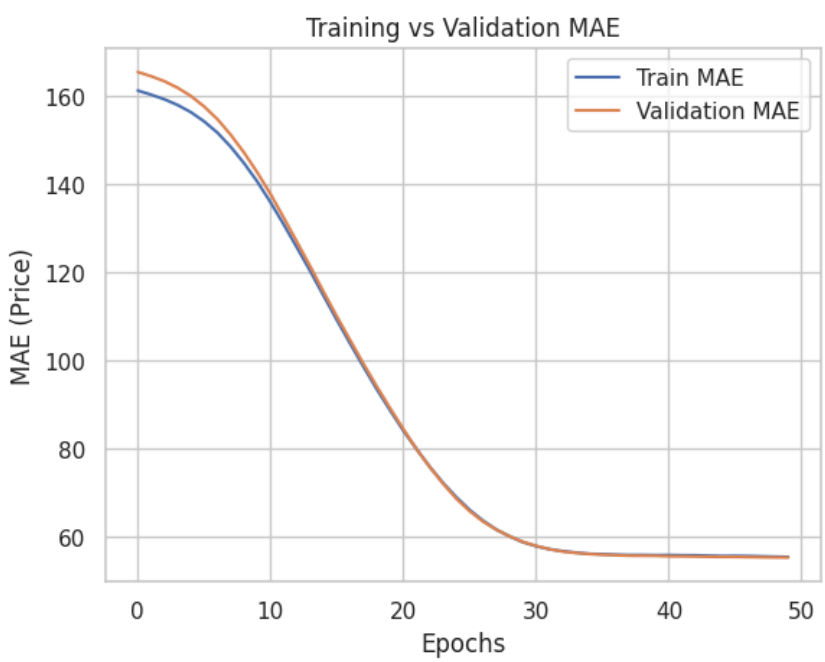
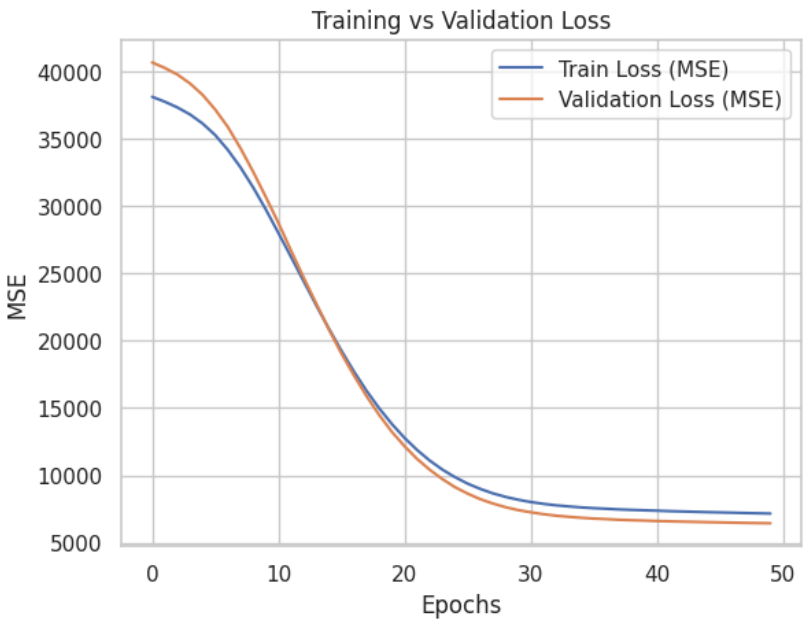


Figure 6:



Based on the results of section 3, it seems that the model underfit. An R^2 of 0.47 means the model explains only 47% of the variance in listing prices, leaving over half unexplained. This is not very good and leaves a lot of room for improvement. The training and validation loss curves converge closely together and flatten early, telling me that the model is not complex enough to capture the underlying patterns. If the model were overfitting, we would expect to see the training loss continue to decrease while the validation loss starts increasing, creating a visible gap between the two curves. Instead, both curves plateau at a similar high level, which confirms the model is too simple for the complexity of the data.

4.1) Suggested Improvements

A possible improvement that comes to mind right off the bat is adding more hidden layers or increasing the number of nodes. Since it seems that the model is not complex enough and is having a hard time with the complex relationships, adding more layers or increasing the number of nodes would give it a better chance of doing so. I also think the model would benefit from more than 50 epochs due to this outcome, maybe closer to 100 would be more beneficial. Additionally, I think including more features like location or property type could help, but that would also make the model more complex so we have to deal with that issue first. Alongside that as I mentioned earlier some future engineering like combining some variables and doing log transformation would also help the models overall performance by addressing skewing and possible redundancy.

Generative AI Usage:

I used generative AI in this written report to help me write a concise introduction and mainly to help explain some topics like mae and mse. In the code I used it to help me debug some errors I got and I used it to help me interpret the visualizations and findings so I could genuinely understand what they meant and write about them with my own words.